

Python Programming Solutions

Concise 7-Mark Explanations (Tabular Format)

Q1 (A) Explain the working of input() function and eval() function in brief alongwith an example. [7 Marks]

1. The input() Function

The `input()` function is used to take user input from the keyboard during program execution. It halts the program until the user enters data and presses Enter.

- **Key trait:** It *always* reads and returns the input as a **string (`str`)**, regardless of whether the user types numbers or characters.
- Type casting (e.g., `int()`) is required to use the input in mathematical operations.

```
# Example of input()
age = input("Enter your age: ") # User enters 20
print(type(age))                # Output: <class 'str'>
print("Next year, age is:", int(age) + 1) # Cast to int for math
```

2. The eval() Function

The `eval()` function takes a string, parses it, and executes it as a Python expression. It essentially evaluates a string as if it were actual Python code.

- **Key trait:** When combined with `input()` , it automatically detects and converts the entered string into the correct Python datatype (like lists, dicts, or numbers).

```
# Example 1: Evaluating a string expression
result = eval("5 * 4 + 10")
print(result) # Output: 30

# Example 2: Dynamic data type from user
data = eval(input("Enter a list: ")) # User enters: [1, 2, 3]
print(type(data))                  # Output: <class 'list'>
```

Q1 (B) What are Datatypes in Python? Explain each datatype alongwith an example. [7 Marks]

A **Datatype** represents the kind of value a variable holds, dictating the operations that can be performed on it. Python is dynamically typed, meaning types are inferred automatically at runtime.

Category	Datatype	Description	Example
Numeric	Integer (<code>int</code>)	Whole numbers without decimals.	<code>age = 25</code>
	Float (<code>float</code>)	Real numbers with decimal points.	<code>price = 99.99</code>
	Complex	Numbers with real and imaginary parts.	<code>z = 3 + 4j</code>
Sequence	String (<code>str</code>)	Immutable sequence of characters.	<code>name = "Python"</code>
	List (<code>list</code>)	Ordered, mutable collection of items.	<code>colors = ["red", "blue"]</code>
	Tuple (<code>tuple</code>)	Ordered, immutable collection of items.	<code>points = (10, 20)</code>

Mapping	Dict (<code>dict</code>)	Unordered collection of key-value pairs.	<code>user = {"id": 1}</code>
Set	Set (<code>set</code>)	Unordered collection of unique items.	<code>ids = {1, 2, 3}</code>
Boolean	Bool (<code>bool</code>)	Represents True or False.	<code>is_active = True</code>

Q2 (A) Explain the importance of comments in Python Programming. Demonstrate with examples how to use single-line and multi-line comments effectively. [7 Marks]

Importance of Comments

- **Readability:** They explain complex logic, making the code understandable for other developers.
- **Debugging:** Developers can temporarily "comment out" code to test for errors without deleting lines.
- **Documentation:** Docstrings can be used by automated tools to generate software manuals.

Types of Comments

Type	Syntax / Method	Usage
Single-Line	Uses the hash symbol <code>#</code>	For brief notes on a single line or inline with code.
Multi-Line	Triple quotes <code>"""</code> or multiple <code>#</code>	For detailed explanations, module descriptions, or function docstrings.

```
"""
This is a multi-line comment (Docstring).
It explains what the below script is intended to do.
"""

# This is a single-line block comment
base_price = 100

final_price = base_price * 0.9 # This is an inline single-line comment
```

Q2 (B) Describe various types of tokens in Python. Explain each token type with an appropriate example. [7 Marks]

A **Token** is the smallest individual unit in a Python program. The interpreter breaks source code into these meaningful chunks during lexical analysis.

Token Type	Description	Examples
Keywords	Reserved words with special meaning to the interpreter. Cannot be used as variable names.	<code>if</code> , <code>def</code> , <code>for</code> , <code>True</code> , <code>return</code>
Identifiers	User-defined names for variables, functions, classes, etc. Must start with a letter or underscore.	<code>age</code> , <code>student_name</code> , <code>_count</code>
Literals	Raw data values directly assigned to variables (constants).	<code>100</code> (Int), <code>"Hello"</code> (String), <code>3.14</code> (Float)
Operators	Symbols that trigger arithmetic, logical, or relational computations.	<code>+</code> , <code>-</code> , <code>==</code> , <code>=</code> , <code>and</code>
Punctuators	Delimiters used to structure code, group expressions, or define syntax.	<code>()</code> , <code>[]</code> , <code>{}</code> , <code>:</code> , <code>,</code>

Example Analysis:

```
total = 50 + 10
```

```
# total -> Identifier  
# =      -> Operator (Assignment)  
# 50     -> Literal (Numeric)  
# +      -> Operator (Arithmetic)  
# 10     -> Literal (Numeric)
```

**Q3 (A) Explain various arithmetic operators available in Python.
Elaborate alongwith an example. [7 Marks]**

Arithmetic operators are used to perform mathematical operations on numeric values (operands).

Operator	Name	Description	Example (a=10, b=3)
+	Addition	Adds values on either side of the operator.	$a + b = 13$
-	Subtraction	Subtracts right hand operand from left hand operand.	$a - b = 7$
*	Multiplication	Multiplies values on either side of the operator.	$a * b = 30$
/	Division	Divides left hand operand by right hand operand (always returns a float).	$a / b = 3.333...$
//	Floor Division	Division that results into whole number adjusted to the left in the number line.	$a // b = 3$
%	Modulus	Divides left hand operand by right hand operand and returns the remainder.	$a \% b = 1$

**	Exponentiation	Calculates the exponential (power) of a value.	<code>a ** b = 1000</code>
----	----------------	--	----------------------------

```

a = 10
b = 3
print("Floor Division:", a // b) # Output: 3
print("Modulus:", a % b)         # Output: 1
print("Exponent:", a ** b)       # Output: 1000

```

Q3 (B) Distinguish between break statement and continue statement alongwith an example. [7 Marks]

Feature	break statement	continue statement
Functionality	Terminates the execution of the entire loop immediately.	Skips the remaining statements in the current iteration and moves to the next iteration.
Execution Flow	Control jumps to the first statement immediately <i>after</i> the loop.	Control jumps back to the loop condition (or iterator) at the <i>beginning</i> of the loop.
Common Use Case	Exiting early when a search target is found or an error condition is met.	Skipping invalid or unwanted data during processing without stopping the loop.

```

# Example of 'break'
for i in range(1, 6):
    if i == 3:
        break # Loop stops completely when i is 3
    print(i, end=" ")
# Output: 1 2

print("\n---")

```

```
# Example of 'continue'
for i in range(1, 6):
    if i == 3:
        continue # Skips printing 3, but loop continues
    print(i, end=" ")
# Output: 1 2 4 5
```

Q4 (A) Write a program in Python to check whether the given number is odd or even. [7 Marks]

Logic: A number is considered even if it is perfectly divisible by 2 (meaning the remainder is 0). We can determine this using the modulus operator (%).

```
# Python program to check if a number is even or odd

# 1. Take integer input from the user
num = int(input("Enter a number to check: "))

# 2. Use the modulus operator to check the remainder
if num % 2 == 0:
    print(f"{num} is an Even number.")
else:
    print(f"{num} is an Odd number.")
```

Q4 (B) Describe various decision-making statements in Python. Design a Python program using nested if-else block. [7 Marks]

Decision-Making Statements

Decision-making statements allow a program to execute different blocks of code based on whether a specific condition evaluates to True or False.

- **if statement:** The simplest decision-making statement. It executes a block of code *only* if the given condition is true.
- **if-else statement:** Provides an alternative. If the `if` condition is true, its block runs. Otherwise, the `else` block runs.
- **if-elif-else statement:** Used for checking multiple conditions sequentially. It prevents deep nesting when checking many independent possibilities.
- **Nested if-else :** Placing an `if` or `else` statement *inside* another `if` or `else` block. It is used to evaluate complex, multi-level conditions.

Nested if-else Program Example

Below is a program that checks if a number is positive, negative, or zero. If it is positive, it further checks if it is even or odd using nesting.

```
# Program using nested if-else statements
number = int(input("Enter a number: "))

if number >= 0:
    # Outer 'if' is true, entering nested block
    if number == 0:
        print("The number is Zero.")
    else:
        print("The number is Positive.")

        # Second level of nesting
        if number % 2 == 0:
            print("It is also an Even number.")
        else:
            print("It is also an Odd number.")
else:
    # Outer 'if' is false
    print("The number is Negative.")
```